Power Systems Agent Benchmark: исполняемое оценивание ИИ-агентов на задачах электроэнергетики

Sergei Trashchenkov trashchenkov@gmail.com · ORCID 0000-0001-8786-8336

Черновик препринта, версия 2 (июнь 2026). Русскоязычная версия; первоисточник — английская версия для arXiv (eess.SY; cs.AI): `paper_en_v2.md`*. Аннотированная библиография:* `bibliography.md`*; BibTeX:* `references.bib`*.*

Аннотация

Исполняемое оценивание (executable evaluation) — когда агент совершает действие, а его последствия проверяются программой, а не оцениванием его текста — стало стандартным способом оценки ИИ-агентов, использующих инструменты (tool-using AI agents), в программной инженерии и смежных областях. Электроэнергетика, напротив, едва начала осваивать агентный ИИ (agentic AI): применение языковых моделей в этой отрасли по-прежнему сводится в основном к поиску и текстовым ответам на вопросы, агенты, действующие в энергосистеме, существуют лишь как несколько академических прототипов, а оценивание, соответственно, касается текста, который выдаёт модель, а не последствий её действия. Мы переносим исполняемое оценивание на агентов электроэнергетики. В Power Systems Agent Benchmark агент получает структурированную задачу и возвращает структурированное решение; затем детерминированный оценщик (evaluator) пересчитывает соответствующие инженерные величины, проверяет эксплуатационные ограничения и возвращает флаг допустимости (feasibility flag), нормированный балл (normalized score) и явный перечень нарушений (violations). Бенчмарк охватывает 41 семейство задач (task families) в восьми областях электроэнергетики — потокораспределение и анализ сети, короткие замыкания и релейная защита, устойчивость и соответствие сетевому кодексу, распределённые ресурсы и накопители, диспетчеризация микросетей, надёжность и восстановление, качество электроэнергии и стандарты, прогнозирование — каждое опирается на цитируемый источник и инженерный стандарт. Помимо публичного набора (public set), скрытые случаи (held-out) не распространяются как данные, а синтезируются по запросу генератором для каждого семейства из приватного зерна (private seed), что даёт устойчивое к загрязнению (contamination-resistant) оценивание, конструкция которого открыта для проверки, а конкретные экземпляры остаются приватными. Поскольку протокол запуска не зависит от конкретного агента (agent-agnostic), бенчмарк служит общей площадкой для сравнения языковых моделей и агентных обвязок (agent harnesses) и для изучения поведения агентов на физически обоснованных задачах; в качестве эталонного оценивания мы рассматриваем трёх агентов командной строки; на нынешнем компактном уровне сложности сильнейшие уже набирают близко к максимуму, тогда как меньшая открытая модель отстаёт, а результаты на публичном и скрытом наборах согласуются. Оценщики представляют собой компактные детерминированные суррогаты, и контракт задачи устроен так, что их внутреннюю реализацию впоследствии можно заменить проверками на основе симулятора, не меняя того, как задачи ставятся и решаются.

1. Введение

Языковые модели за короткое время прошли путь от систем, порождающих текст, к системам, совершающим действия. Получив доступ к инструментам, модель может прочитать структурированные данные, вызвать внешнюю программу, изучить результат и решить, что делать дальше; помещённая в такой цикл, она становится тем, что сейчас принято называть агентом (agent). Этот сдвиг заметнее всего в программной инженерии, где агенты решают задачи в реальных репозиториях кода, и в бизнес-операциях, где они ведут клиентские процессы, обращаются к базам данных и выполняют

многошаговые процедуры через вызовы инструментов и API. Привлекательность во всех этих случаях одна и та же: агент способен выполнить работу, которая прежде требовала человека, переводящего намерение в последовательность вызовов инструментов.

Естественно спросить, переносится ли та же способность на специализированные инженерные области, где работа подчиняется не соглашениям, а физическим законам. Электроэнергетические системы — показательный и требовательный пример. Инженерные решения здесь — оценка тока короткого замыкания, выбор уставки релейной защиты, планирование генерации при ограничениях по безопасности, скрининг сети на аварийные ситуации — опираются на теорию цепей, на принципы устойчивости и релейной защиты и на плотный свод стандартов. Способны ли языковые агенты выполнять такую работу — открытый исследовательский вопрос, и притом активный: недавние обзоры фиксируют быстрый приток агентных и LLM-методов в анализ энергосистем [llmpowersurvey2025, agenticpowersurvey2025], и начали появляться первые прототипы [gridmind2025, xgridagent2025]. Эта активность пока почти полностью экспериментальная, а не эксплуатационная, что само по себе делает вопрос об измерении прогресса насущным.

Измерять прогресс труднее, чем кажется на первый взгляд, потому что правильность в электроэнергетике — это физическое свойство, а не текстовое. Предложенную уставку защиты можно красноречиво обосновать, и она всё равно окажется неселективной; диспетчерский план может быть внутренне непротиворечив и при этом превышать пределы генератора; план восстановления может хорошо читаться и обесточить не тот фидер. Ошибки такого рода невидимы человеку, бегло просматривающему пояснение, и языковой модели, которой поручили его оценить, потому что это не ошибки изложения — это нарушения физики сети и эксплуатационных пределов, и проявляются они лишь тогда, когда предложенное решение пересчитывают применительно к случаю. Два наиболее распространённых в агентных исследованиях способа оценивания именно по этой причине здесь плохо подходят: ответы на вопросы по фиксированному эталонному ответу не вмещают множества допустимых решений, которые допускает инженерная задача, а оценивание свободного отчёта второй моделью [zheng2023judging] вознаграждает гладкое обоснование даже тогда, когда лежащее в его основе решение несостоятельно, — склонность, усиливаемая документированными смещениями таких судей [wang2023fairevaluators].

В программной инженерии и работе с инструментами эту трудность преодолели исполняемым оцениванием: вывод агента не оценивают как текст, а применяют и проверяют программой. SWE-bench [jimenez2024swebench] прогоняет собственный набор тестов репозитория против патча агента; τ-bench [yao2024taubench] сравнивает конечное состояние базы данных с размеченной целью; terminal-bench [terminalbench2025] проверяет эффект команд агента в живой командной оболочке. Общий для них принцип — судить по итогу действия, а не по его описанию — это ровно то, что нужно оцениванию в электроэнергетике, и в сколько-нибудь широком масштабе к этой области он не применялся.

Эта работа делает именно так. Мы представляем Power Systems Agent Benchmark, в котором каждая задача даёт агенту структурированные входные данные и требуемую схему вывода; агент возвращает структурированное решение; а детерминированный оценщик пересчитывает соответствующие инженерные величины, проверяет жёсткие ограничения (hard constraints) и возвращает флаг допустимости, нормированный балл в единичном интервале и явный перечень нарушений. Текстовое пояснение агента сохраняется, но в оценку не входит. Публичная часть охватывает 41 семейство задач в восьми областях электроэнергетики, каждое из которых опирается на цитируемый источник и инженерный стандарт. Скрытая часть не распространяется как данные: генератор порождает скрытые случаи по запросу из приватного зерна, так что устройство скрытого набора открыто для проверки, а его конкретные экземпляры остаются неизвестны оцениваемым.

Этот бенчмарк — наш основной вклад, и его удобнее всего понимать как три вещи сразу. Во-первых, это широкая исполняемая проверка компетентности в электроэнергетике: 41 семейство задач, охватывающее восемь областей отрасли, каждое поставлено через единый контракт и проверяется детерминированным оценщиком, который пересчитывает лежащую в основе физику, так что агент измеряется тем, что делают его решения, а не тем, как он их описывает. Во-вторых, это устойчивое к загрязнению оценивание: вместо того чтобы поставлять фиксированный скрытый набор, который стареет и утекает, мы даём генератор для каждого семейства, синтезирующий физически непротиворечивые, проверенные на разрешимость случаи из приватного зерна, сохраняя устройство скрытого набора проверяемым, а его экземпляры — приватными. И в-третьих, это площадка, не зависящая от конкретного агента: поскольку участие сводится к чтению файла задачи и записи файла решения, тот же бенчмарк может сравнивать разные языковые модели и разные агентные обвязки в рамках одного протокола и служить основой для изучения поведения агентов на физически обоснованных задачах. Мы сопровождаем бенчмарк эталонным оцениванием трёх агентов командной строки и по ходу его проведения сообщаем методологическое наблюдение — что провал, общий для всех агентов, является надёжным признаком дефектной задачи или оценщика, — достаточно полезное, чтобы рекомендовать его само по себе (раздел 8.2). Мы позиционируем это как v0: широкий, устойчивый к загрязнению, не зависящий от конкретного агента исполняемый прототип на 41 компактном семействе задач — не как притязание на первенство относительно более ранних доменных работ (раздел 2) и не как меру компетентности промышленного масштаба (раздел 9).

2. Связанные работы

Наша работа опирается на три направления исследований, и её проще всего расположить по тому, где она следует за каждым из них и где отходит от него.

Первое — исполняемое оценивание агентов. SWE-bench [jimenez2024swebench] и более ранняя конструкция pass@k из Codex/HumanEval [chen2021codex] установили, что вывод агента можно судить, исполняя его; τ-bench [yao2024taubench] обобщил это на взаимодействие с инструментами и пользователем, сравнивая конечные состояния, и предложил pass^k как меру надёжности; terminal-bench [terminalbench2025] и MLE-bench [chan2024mlebench] распространяют идею на работу в терминале и инженерию машинного обучения, а широкие наборы вроде AgentBench [liu2024agentbench], GAIA [mialon2023gaia] и WebArena [zhou2024webarena] проверяют общее использование инструментов. Мы перенимаем эту парадигму; наше отличие — в охвате, поскольку ни один из перечисленных не нацелен на инженерию энергосистем и её физические ограничения.

Второе направление касается того, *как* оценивать, и именно поэтому мы пересчитываем, а не судим. В конечном счёте важен эффект действий агента, а не гладкость его рассказа о них, и для проверки эффектов сильнейший доступный инструмент — детерминированная проверка. Это не новая идея — на ней держатся тестирование программ и спортивное программирование, где решение принимается только если оно проходит фиксированный набор тестов, — и везде, где задача допускает такую проверку, как допускает каждая задача в этом бенчмарке, её следует предпочесть: она точна, воспроизводима и невосприимчива к смещениям, сопровождающим судью на основе языковой модели, — смещениям позиции, длины и самопредпочтения [zheng2023judging, wang2023fairevaluators], которые наиболее разрушительны как раз тогда, когда неверный ответ гладок. Оценивание на основе модели сохраняет роль там, где детерминированная проверка неосуществима, и недавние работы распространяют его с отдельных ответов на целые траектории агента под названием *агент-как-судья* (agent-as-a-judge) [zhuge2024agentjudge, aisjudgeais2025]. Мы ожидаем, что по мере роста бенчмарка и включения сценариев, слишком открытых для оценщика в замкнутой форме, оценивание такого рода — применяемое к действиям и траекториям, а не к тексту — станет уместным, и оставляем

этот путь открытым; для представленных здесь задач достаточно детерминированного пересчёта, а обоснование агента сохраняется лишь как диагностика.

Третье направление — предметно-специфическое. Обзоры [llmpowersurvey2025, agenticpowersurvey2025] фиксируют быстрый рост LLM- и агентных методов в энергосистемах при отсутствии устоявшегося способа оценивания. Ближайшая к нашей работа — PowerAgentBench [poweragentbench], часть активно развиваемой экосистемы PowerAgent в Harvard SEAS [zhang2025poweragent]. Мы перенимаем тот же базовый замысел, который она использует, — разделение на публичный и скрытый наборы с оценщиком, пересчитывающим физическую достоверность, — и не претендуем на его новизну: PowerAgentBench — более ранний пример такого подхода в этой области, а устойчивое к загрязнению оценивание на скрытом наборе к настоящему времени стало широко принятой идеей [contaminationsurvey2025, white2024livebench]. ElecBench [zhou2025elecbench] оценивает сценарии диспетчеризации преимущественно через текст и судей, а PFBench [pfbench2025] применяет программную проверку, но только к потокораспределению. Появляющиеся ныне прототипы агентов — GridMind [gridmind2025], X-GridAgent [xgridagent2025], Grid-Mind CIA [gridmindcia2026], PFAgent [pfagent2026] — и есть предполагаемые объекты бенчмарка такого рода. Нашу работу отличает не само разделение, а его акценты: широта по многим семействам задач, скрытые случаи, порождаемые из зерна, а не хранимые, и явная процедура контроля качества.

Ещё два корпуса работ влияют на конкретные решения. Среды управления сетью на основе обучения с подкреплением, прежде всего Grid2Op/L2RPN [marot2020l2rpn, marot2021retrospective], дают зрелую постановку для последовательного управления топологией и построены на устоявшихся симуляторах энергосистем — pandapower [thurner2018pandapower], MATPOWER [zimmerman2011matpower], OpenDSS [dugan2011opendss], PowerModels/PGLib [coffrin2018powermodels, babaeinejadsarookolaee2019pglib], ANDES [cui2021andes], PyPSA [brown2018pypsa], — годами оттачиваемых для расчёта физики энергосистем. Мы решаем иную задачу — одношаговые инженерные решения по многим подзадачам в рамках контракта, не зависящего от агента, — и наши нынешние оценщики этих симуляторов не используют вовсе: каждый представляет собой компактную, самодостаточную детерминированную проверку. Мы называем их лишь как естественный бэкенд для будущих версий оценщиков более высокой точности, а не как нынешнюю зависимость. Наконец, литература о целостности бенчмарков влияет на устройство скрытого набора: поскольку загрязнение обучающими данными завышает результаты [contaminationsurvey2025, statictodynamic2025], динамические и обновляемые бенчмарки стали стандартным ответом [kiela2021dynabench, white2024livebench, jain2024livecodebench], а генератор на основе зерна — это механизм предотвращения того же семейства. Раннер следует предписаниям [kapoor2024agentsmatter] по стоимости, воспроизводимости и разделению классов провалов и учитывает пределы best-of-N (выбор лучшего из N попыток), охарактеризованные в [brown2024monkeys].

3. Требования к устройству

Система формируется небольшим набором требований, которые изложены здесь в самом начале, потому что большая часть последующего — их следствие; для каждого мы указываем, где оно реализовано.

1. **Исполняемая, физическая проверка.** Решение должно проверяться исполнением кода, а не сопоставлением строк, и этот код должен пересчитывать соответствующие физические величины и проверять эксплуатационные ограничения, а не сравнивать с хранимым текстом. Остальные требования по большей части вытекают из этого (контракт дан в разделе 4.1, оценщики — в разделе 6).

2. **Защищённое от утечки разделение на публичный и скрытый наборы.** Публичные случаи должны поддерживать разработку, а скрытые — честное сравнение; что особенно важно, скрытые ответы должны исключаться по построению, а не по регламенту, так чтобы никакое изучение опубликованного материала их не раскрывало (раздел 4.4).
3. **Независимость от агента.** Поскольку смысл в том, чтобы сравнивать модели и обвязки на равных, запуск агента не должен зависеть от какого-то одного из них — участие сводится к чтению файла задачи и записи файла решения (раздел 4.2).
4. **Происхождение и машиночитаемые результаты.** Каждая задача должна объявлять, откуда она происходит, с уровнем достоверности (раздел 5.1), а каждый результат должен быть машиночитаемым — флаг допустимости, балл и явный перечень нарушений (разделы 4.1 и 6), — чтобы прогоны можно было агрегировать и проверять без чтения текста кем бы то ни было.
5. **Запас для роста.** Бенчмарк должен иметь возможность становиться требовательнее без нарушения совместимости, так чтобы замена проверки в замкнутой форме на проверку на основе симулятора меняла внутреннюю реализацию оценщика, оставляя форматы задачи и решения нетронутыми (раздел 4.3).
6. **Контроль качества спецификаций.** Наконец, поскольку собственные задачи и оценщики бенчмарка сами могут быть ошибочны, он должен давать средство отличить дефект в своих спецификациях от подлинного различия в способностях агентов (показано в разделе 8.2).

4. Архитектура

4.1 Контракт артефактов

Бенчмарк организован вокруг единого линейного контракта (рис. 1): описание задачи выдаётся агенту, агент порождает структурированное решение, оценщик потребляет решение, и выдаётся результат оценивания. JSON-схема фиксирует каждый из трёх артефактов. Задача несёт свою идентичность и семейство, публичные входные данные, схему, которой должно следовать решение, жёсткие ограничения, которые будут проверяться, метрики, происхождение и двуязычный блок представления. Решение несёт структурированный ответ агента и необязательное обоснование. Результат оценивания несёт флаг допустимости, балл, перечень нарушений и любые подтверждающие свидетельства. Поскольку форматы зафиксированы независимо от того, как задача решается или проверяется, агенту и оценщику не нужно разделять ничего, кроме этих схем.



Рис. 1: Рис. 1. Контракт бенчмарка. Задача ставится перед любым агентом командной строки, который записывает структурированное решение; детерминированный оценщик пересчитывает результат и выдаёт машиночитаемый вердикт.

Сокращённый пример задачи о коротком замыкании иллюстрирует форму:

```
{
  "family": "three_phase_short_circuit",
  "public_inputs": {"short_circuit_case": {
    "base_mva": 100.0, "voltage_kv": 10.5,
    "voltage_factor_cmax": 1.1, "voltage_factor_cmin": 1.0,
    "positive_sequence_impedances_pu": [ /* ... */ ],
```

```
    "fault_resistance_pu": 0.005}},
  "submission_schema": {
    "ik_max_ka": "max initial symmetrical current (kA)",
    "ik_min_ka": "min current with fault resistance (kA)", "...": "..."},
  "hard_constraints": [
    "maximum current uses c_max over a bolted fault",
    "values match the independent positive-sequence calculation"],
  "metrics": [{"name": "feasible", "direction": "boolean", "...": "..."},
              {"name": "score", "direction": "maximize", "...": "..."}],
  "source_provenance": [{"title": "IEC 60909 short-circuit calculation",
                         "confidence": "high", "...": "..."}]
}
```

4.2 Запуск, не зависящий от агента, и надёжный раннер

Чтобы сравнение между агентами было осмысленным, *раннер* (runner) — собственный механизм бенчмарка, который запускает каждого агента (а тот сам является обвязкой (harness) из модели, цикла и инструментов) на каждой задаче и оценивает то, что он возвращает, — должен обладать тремя свойствами помимо простого вызова агента. Первое — изоляция: агент не должен иметь возможности прочитать оценщик или какой-либо хранимый ответ. Поэтому каждая задача упаковывается с удалёнными ключом-ответом и внутренностями оценщика и исполняется в песочнице (sandbox), не имеющей пути обратно к собственному коду бенчмарка. Это важно, потому что агент с автономным доступом к файловой системе в принципе способен добраться до оценщика, оказавшегося в пределах досягаемости, и оценить себя по нему; письменная инструкция этого не делать не является защитой, поскольку ничто не вынуждает агента её соблюдать. Поэтому изоляция должна быть структурной, а не инструктивной.

Второе свойство — устойчивость к некорректному выводу. Агент может вернуть решение, синтаксически правильное, но бессмысленное — пустые значения, текст там, где требуется число, вовсе неверную структуру, — и оценщик должен трактовать это как обычное неудачное решение, возвращая «недопустимо» с перечнем проблемных полей, а не возбуждать исключение, останавливающее весь прогон. Без этого свойства одно некорректное решение способно оборвать многозадачное оценивание на полпути и оставить неполную, несопоставимую запись.

Третье свойство — принципиальное разделение классов провалов. Прогон может завершиться неудачей по причинам, не имеющим отношения к компетентности агента, — кратковременный сбой провайдера, разрыв соединения, исчерпанная квота использования, — и смешение их с подлинными провалами загрязняет измерение. Раннер классифицирует каждый провал и перезапускает только те, что относятся к инфраструктуре, и никогда — те, в которых агент выдал плохое или отсутствующее решение в исправной среде; перезапуск последних незаметно превратил бы бенчмарк в поиск best-of-N и завысил бы каждый балл. Каждый результат фиксирует, сколько попыток потребовалось и почему провалились предыдущие, а прогон сообщает свои метрики и с инфраструктурными перезапусками, и без них, так чтобы надёжность провайдера никогда не принимали за способность агента.

4.3 Лестница зрелости оценщиков

Оценщикам дозволено созревать на месте вдоль лестницы (maturity ladder): от суррогата в замкнутой форме или на основе графов, через детерминированную инженерную модель, к проверке, опирающейся на полноценный симулятор энергосистемы, и в конечном счёте к стохастическому или круп-

номасштабному набору сценариев. Представленные здесь оценщики занимают первые две ступени, и в пределах этих ступеней они двух видов. Примерно половина вычисляет инженерную величину в замкнутой форме по методу из стандарта или учебника — токи короткого замыкания и токи по симметричным составляющим, матрицу узловых проводимостей, тепловую модель трансформатора, пределы по гармоникам и по напряжению — и точны для поставленной компактной задачи. Остальные оценивают решение над намеренно упрощённой моделью — линейная чувствительность напряжения вместо полного расчёта в переменном токе или оптимизация над компактной моделью диспетчеризации либо восстановления — и приближённы по построению; граница не всегда чёткая, поскольку некоторые точно решают линеаризованную модель. Ни один пока не вызывает полноценный симулятор энергосистемы — это следующая ступень. Контракт раздела 4.1 — это то, что делает лестницу применимой: поскольку форматы задачи и решения зафиксированы независимо от того, как вычисляется проверка, внутренний шаг замены, скажем, линеаризованного суррогата потокораспределения на полное решение в переменном токе повышает точность оценщика, не меняя ничего из того, что видит агент. Лестница описывает, как вычисляется проверка, но контракт симметричен по инструментам, которые он допускает: ничто не мешает агенту со своей стороны вызвать тот же класс программ, чтобы получить решение, — возможность, которую нынешние задачи ещё не задействуют, а будущие задействуют (раздел 10).

4.4 Скрытый набор как генератор

Бенчмарк вообще не хранит свои скрытые случаи. Вместо файлов скрытых данных он предоставляет генератор: набор детерминированных, учитывающих физику возмущений, по одному на семейство. Скрытый случай порождается из публичного случая перерисовкой его числового содержимого — параметров сети, рядов погодных данных, профилей нагрузки, места повреждения вместе с показаниями индикаторов и жалобами потребителей, которые оно подразумевает, — под приватным случайным зерном, и бросок принимается только если эталонный решатель (reference solver) семейства по-прежнему решает его, оценщик подтверждает решение допустимым, и никакое малое возмущение этого решения не даёт более высокого балла, — сохраняя эталонное решение тем оптимумом, относительно которого оценивается случай (раздел 6), — иначе следует детерминированная перерисовка; раздел 8.3 эмпирически подтверждает, что это даёт решаемые, разнообразные скрытые экземпляры в масштабе. Тот, кто ведёт таблицу лидеров (leaderboard), держит зерно приватным; любой другой может прочитать в точности, как устроены скрытые случаи, не узнавая конкретных экземпляров какого-либо тура. Это делает скрытый набор — обычно самый подверженный утечке компонент бенчмарка — одновременно проверяемым по механизму и приватным по содержанию. Та же дисциплина распространяется на записи прогонов оценивания: решение агента для скрытого случая и по-кейсный вывод оценщика каждый раскрывают ответ этого случая, поэтому публикуются только агрегированные сводки прогонов — но никогда по-кейсные скрытые решения, результаты или журналы.

5. Набор задач

Публичный бенчмарк составляют 41 семейство задач, сгруппированные в восемь предметных областей:

1. **Анализ сети и эксплуатационные ограничения** — построение матрицы узловых проводимостей, корректирующая диспетчеризация реактивной мощности для соблюдения пределов напряжения, скрининг аварийных ситуаций (contingency screening) [zhou2019contingency], послеаварийные корректирующие переключения, динамическое определение допустимой токовой нагрузки линии [ieee738].

2. **Короткое замыкание и релейная защита** — расчёт трёхфазного и однофазного на землю короткого замыкания [iec60909, anderson1995faulted], выбор уставок выключателя и реле по уровням токов КЗ, настройка зон дистанционной защиты и согласование максимальной токовой защиты [horowitz2014relaying], выбор сечения кабеля [iec60364552].
3. **Устойчивость, сетевой кодекс и инверторные источники** — критическое время отключения по критерию равных площадей (equal-area criterion) [kundur1994stability], классификация переходной устойчивости, соответствие требованиям по выдерживанию провалов напряжения (fault-ride-through, FRT) [ieee2800, entsoerfg], отклик инвертора с ограничением тока при КЗ, минимальная доля синхронной генерации при ограничениях по инерции и току КЗ.
4. **Распределённые ресурсы, СЭС, электромобили и накопители** — управление Volt-VAR [turitsyn2010voltvar], планирование vehicle-to-grid (V2G) и поддержка напряжения, отклик накопителя на вспомогательные услуги, расчёт приведённой стоимости с учётом «мягких» затрат.
5. **Микросети и диспетчеризация** — экономическая и скользящая по горизонту диспетчеризация, диспетчеризация активной/реактивной мощности в островном режиме, робастная диспетчеризация при неопределённости, выбор состава включённого оборудования (unit commitment) для гидро-теплового парка, выбор состава генерации с обеспечением N-1 [scuccts2020].
6. **Надёжность и восстановление** — изоляция повреждения и восстановление электроснабжения [restoration2020], локализация повреждённого участка, размещение индикаторов повреждения, улучшение показателей надёжности [ieee1366], переключения и отключения нагрузки оператором.
7. **Качество электроэнергии, стандарты, активы и кибербезопасность** — проверка соответствия напряжения стандарту электроснабжения [en50160], классификация событий качества электроэнергии, соответствие пределам по гармоникам [ieee519], тепловая нагрузка трансформатора и старение изоляции [iec600767], выявление недостоверных данных при оценивании состояния [abur2004se], размещение защищённых измерений [liu2011fdi], передача функции ведущей станции в многоагентной схеме регулирования частоты.
8. **Прогнозирование при неопределённости** — прогноз выработки ветра, оценивание доверительного интервала прогноза.

Каждая задача перед включением проверялась на физическую корректность по цитируемому источнику. Каждое семейство несёт документированный источник и уровень достоверности (раздел 5.1), а приложение приводит полный каталог с идентификаторами.

5.1 Происхождение и достоверность

Поскольку набор задач почерпнут из инженерной литературы, а не написан с нуля, каждое семейство фиксирует, откуда взята его постановка и насколько прочно она привязана к этому истоку. Каждая задача несёт запись о происхождении (provenance) — название источника, локатор (цифровой идентификатор объекта или URL для статьи, пункт для стандарта или полную ссылку для учебника) и краткую заметку о том, как источник отображается на исполняемый случай, — вместе с заявленным уровнем достоверности. Уровень достоверности — это намеренное, объявленное суждение, а не измеренная величина, и мы присваиваем его по фиксированной рубрике, чтобы он был воспроизводим:

- **высокая** — задача воспроизводит метод, определённый в опубликованном стандарте или каноническом учебнике, с конкретной ссылкой, и исполняемый случай следует этому методу напрямую (например, задачи о коротком замыкании [iec60909], о нагрузке трансформатора [iec600767] и о соответствии по гармоникам [ieee519] следуют именованным международным стандартам, а задачи о критическом времени отключения [kundur1994stability] и о

недостоверных данных [abur2004se] следуют стандартным учебным процедурам);
- **средняя** — задача привязана к идентифицированному первоисточнику или к повсеместно преподаваемому инженерному методу, выписанному в виде его определяющих уравнений, и случай является верным компактным его экземпляром;
- **низкая** — ни один отдельный первичный документ не закреплён: метод стандартен, но задача является адаптацией в его духе, а не воспроизведением конкретного источника.

По этой рубрике из 41 семейства 11 оценены как высокие, 28 — средние и 2 — низкие; два низких случая (диспетчеризация с учётом неопределённости и минимальная доля синхронной генерации) сохраняют эту оценку, потому что ни один источник, который мы могли проверить, не обосновывает полностью их точную постановку, и мы предпочитаем сказать об этом, а не завышать ярлык. Мы рассматриваем схему как нижнюю границу прозрачности, а не как гарантию точности: она позволяет читателю проследить любую задачу до её истока и с одного взгляда увидеть, насколько прочно она там закреплена, и делает явным, какие задачи являются верными воспроизведениями стандарта, а какие — более вольными адаптациями общепринятой практики.

6. Оценщики и метрики

Каждое оценивание возвращает одни и те же четыре вещи: флаг допустимости, балл в единичном интервале, перечень нарушений, изложенных словами, и предметные свидетельства. Два числа играют разные роли. Каждая задача объявляет набор *жёстких ограничений* — физических и эксплуатационных требований, которым решение должно удовлетворять, чтобы вообще быть приемлемым, таких как соблюдение пределов генератора, удержание каждого напряжения в его диапазоне или сохранение радиальности сети, — и решение *допустимо* (feasible) только если не нарушает ни одного из них. Балл же оценивает качество среди допустимых решений; недопустимое решение получает ноль, как бы близко оно ни выглядело.

Как вычисляется балл, зависит от задачи, но всякий балл сообщается *относительно эталонного оптимума* (reference optimum): он масштабируется так, чтобы оптимальное решение получало ровно единицу, а допустимое, но худшее — долю ниже неё. Это важно, потому что у многих задач нет естественного единичного потолка для их сырой целевой величины — реактивного усилия в квар, стоимости переключений, объёма восстановленной нагрузки, — так что без такого масштабирования даже идеальный ответ получал бы меньше единицы. Эталонный решатель каждого семейства вычисляет этот оптимум и оптимален по построению; решение агента, будучи одним из многих допустимых, не может его превзойти, и если решение всё же получает больше, то эталон не был оптимален на этом случае, и сообщаемый балл ограничивается единицей. Точное вычисление оценивается по числовым допускам; задача управления или оптимизации оценивает разрыв между поданной целевой величиной и оптимумом, так что альтернативный оптимум равной стоимости получает полный балл, а допустимое, но более дорогое решение — частичный; задача классификации или локализации оценивается по точным множествам с перечислением промахов; а задача соответствия стандарту — как «прошло/не прошло» с диагностическими полями.

Задача диспетчеризации микросети делает различие наглядным: её жёсткие ограничения — это баланс мощности, пределы генератора и накопителя и границы заряда; любой график, удовлетворяющий им всем, допустим, и среди допустимых графиков наименее затратный получает ровно единицу, тогда как более дорогой допустимый план получает меньше единицы пропорционально своему разрыву по стоимости.

На уровне целого прогона мы сообщаем два агрегата, которые не обязаны совпадать: *долю допустимых* (feasible rate) — долю задач, чьё решение удовлетворило каждому жёсткому ограничению, —

и *долю идеальных* (perfect rate) — долю набравших ровно единицу (допустимых и, где оценивается, оптимальных). Эти две расходятся всякий раз, когда агент выдаёт приемлемые, но неоптимальные решения, и, поскольку они могут ранжировать агентов по-разному, мы сообщаем обе наряду со средним баллом (mean score).

7. Воспроизводимость

Любое число в этой статье можно воспроизвести из репозитория. Одна команда прогоняет модульные тесты по оценщикам, эталонным решателям, раннеру и генератору; проверяет каждый артефакт; оценивает каждое из 41 публичного образцового решения по отдельности; и прогоняет эталонные решатели по публичному набору, которые возвращают все 41 случай допустимыми. Поскольку балл каждой задачи измеряется относительно её собственного эталонного оптимума (раздел 6), эталонное решение получает единицу по построению, так что проверка подтверждает не то, что задачи лёгкие, а то, что у каждой задачи есть соответствующее контракту, физически непротиворечивое эталонное решение. Один предел такой проверки стоит назвать прямо: эталонный решатель и оценщик, построенные на одних и тех же ядрах, могут разделять одну и ту же ошибку и всё равно согласовываться, поэтому их согласие подтверждает контракт, но само по себе не удостоверяет лежащую в основе оценщика физику. Как частичную независимую проверку этой физики набор оценщиков дополнительно сверяется с golden-кейсами, ответы к которым выведены вручную из определяющих формул, а не из наших же эталонных решателей, — на данный момент три: трёхфазное короткое замыкание (IEC 60909), однофазное на землю методом симметричных составляющих и построение матрицы узловых проводимостей. Непрерывная интеграция прогоняет полную проверку при каждом изменении.

8. Эксперименты

Мы оценили трёх агентов командной строки на идентичном входе задачи — одном и том же `task.json` и одном и том же файле инструкций для каждого агента на каждой задаче — при фиксированном времени ожидания на задачу и политике инфраструктурных перезапусков из раздела 4.2, без какой-либо подгонки агентов под бенчмарк. Каждый агент вызывался через собственный интерфейс командной строки и работал под тем внутренним системным промптом и обвязкой, с которыми он поставляется; мы стандартизируем то, что спрашивает задача, а не то, как агент организует себя для ответа, поскольку эта обвязка — часть того, чем агент является и что бенчмарк сравнивает. Эти трое — Codex (codex-cli 0.139.0) на gpt-5.5 со средним уровнем рассуждения, агент командной строки Cursor (cursor-agent 2026.06.12) на composer-2.5 и открытый агент OpenCode (opencode 1.15.13) на deepseek-v4-flash-free, бесплатном тарифе этой модели через провайдера OpenCode; ниже мы называем их Codex, Cursor и OpenCode. И модели, и эти CLI агентов часто обновляются, поэтому тур, проведённый в июне 2026 года, следует читать как снимок этих конкретных версий, а не как постоянный рейтинг; повторный прогон того же протокола с более поздними выпусками может дать иные результаты.

8.1 Оценивание на публичном и скрытом наборах

Каждый агент пытался решить публичный случай каждого семейства и один свежесгенерированный скрытый случай на семейство из нового приватного зерна, и каждое решение оценивалось одними и теми же детерминированными, нормированными на оптимум оценщиками.

агент	допустимых (публ.)	идеальных (публ.)	средний (публ.)	допустимых (скр.)	идеальных (скр.)	средний (скр.)
Codex	40/41	38/41	0.973	41/41	40/41	0.995
Cursor	40/41	39/41	0.973	41/41	41/41	1.000
OpenCode	39/41	39/41	0.951	40/41	39/41	0.973

Выделяются две вещи, и первая отрезвляюща для бенчмарка: сильнейшие агенты набирают настолько близко к максимуму, что у него остаётся мало места, чтобы их различить. Codex и Cursor почти неотличимы на публичном наборе и по существу решают скрытый, а OpenCode отстаёт лишь незначительно. Поэтому мы не представляем этот тур как резкое ранжирование передовых агентов; на нынешнем компактном уровне сложности у бенчмарка ограниченная способность разделять лучших из них — вопрос, к которому мы возвращаемся в разделе 9.

Во-вторых, два набора согласуются — каждый агент набирает на свежесгенерированных скрытых случаях не меньше, чем на публичных. Признак, которого ожидали бы от загрязнения, — публичные баллы выше скрытых — отсутствует; если уж на то пошло, скрытые броски оказались чуть легче. Мы не утверждаем наличия внутреннего разрыва в сложности между наборами, и обоснование генератора — проспективное: опубликованный набор стареет по мере того, как модели обучаются на нём, и именно генератор сохраняет доступной незагрязнённую выборку.

То различение, которое бенчмарк всё же даёт, проявляется на нижнем конце диапазона способностей и через общие слепые зоны, а не через зазоры между лидерами: OpenCode полностью проваливает задачу диспетчеризации микросети там, где другие справляются, а одну задачу планирования накопителя все агенты упускают одинаково — это подлинный пробел в способностях, подтверждённо не являющийся дефектом спецификации (раздел 8.2).

8.2 Отличие способности от спецификации

Чтение провалов поперёк агентов, а не по каждому агенту в отдельности, оказалось наиболее полезным доступным анализом, потому что провалы разделяются на два вида. Когда агенты проваливают задачу по-разному — один нарушает предел напряжения там, где другие его удерживают, один возвращает неверный вердикт по выдерживанию провала, один путает знак, — расхождение является сигналом, и именно его таблица лидеров должна вознаграждать. Но когда каждый агент проваливает задачу одинаково, подозревать в первую очередь следует саму задачу, а не агентов. Использование этого паттерна как диагностики при сборке вскрыло ряд дефектов спецификаций — несоответствия между описанием поля и значением, которого ожидал оценщик, неназванные соглашения и излишне строгие сравнения, отвергавшие допустимые ответы, — каждый из которых подтверждался исправлением спецификации и наблюдением за тем, как агенты восстанавливаются. Паттерн — это подозрение, а не приговор: единодушный провал иногда оказывается общей слепой зоной, а не дефектом, поэтому его следует подтверждать осмотром. Мы предлагаем это правило — считать единодушный провал вероятным дефектом спецификации или оценщика, а затем подтверждать или отклонять его — как рутинный шаг контроля качества для исполняемых бенчмарков.

Самый значимый случай указывал не на задачу, а на оценщик, и его стоит изложить полностью, поскольку это самое ясное свидетельство ценности метода, какое у нас есть. На одном скрытом случае по надёжности, касавшемся локализации повреждённого участка, все три агента определили один и тот же повреждённый участок — вывод, согласующийся и с показаниями индикаторов тока КЗ, и с картиной жалоб потребителей; оценщик же требовал другого участка, который на самом деле про-

тиворечил собственным показаниям индикаторов этого случая. Агенты были правы, а оценщик ошибался. Причиной была ошибка на единицу (off-by-one) в модели индикаторов оценщика: он поместил узел ниже повреждённого участка на путь тока КЗ, так что индикатор в этом узле, по предсказанию, регистрировал бы ток КЗ, которого на самом деле не увидел бы, тогда как индикатор регистрирует ток КЗ лишь когда повреждение лежит строго ниже него по сети. Что делает это большим, чем исправленная ошибка, — то, что собственные предохранители бенчмарка не смогли бы её поймать. Эталонный решатель, используемый для самопроверки, выведен из той же функции, что и оценщик, поэтому решатель и оценщик согласовывались друг с другом, и публичный образец проходил; ошибка была невидима изнутри и стала видна лишь потому, что три независимых агента привлекли верную физику. Иначе говоря, исполняемый бенчмарк может таить латентную предметную ошибку в самом коде, определяющем правильность, а матрица агентов — средство её найти; эту осторожность мы распространили бы на любой бенчмарк, чьи эталонные ответы выведены из его оценщика.

8.3 Валидация генератора скрытого набора

Механизм скрытого набора из раздела 4.4 полезен лишь если генератор надёжно выдаёт решаемые, разнообразные экземпляры, а не почти-копии публичного кейса. Мы проверили это напрямую, сгенерировав по восемь скрытых вариантов каждого семейства из фиксированного валидационного зерна и оценив каждый эталонным решателем. Все 328 вариантов (41 семейство × 8) были получены, не исчерпав бюджет перерисовок, и оказались допустимыми, так что правило приёмки — принять бросок только если эталонный решатель его решает и никакое малое возмущение этого решения не даёт более высокого балла — выдерживает масштаб. Каждый вариант отличался от своего публичного кейса, и 40 из 41 семейства дали восемь различных экземпляров; исключение — локализация повреждённого участка — берёт место повреждения из небольшого дискретного множества и потому даёт меньше различных экземпляров (четыре из восьми), что является внутренней границей категориальных задач, а не дефектом генератора. По непрерывному содержимому возмущённые входы варьировались со средним коэффициентом вариации около 0.2, так что варианты — действительно разные задачи. Это валидирует сам генератор: это свойство устройства скрытого набора, а не мера качества агентов, для которой потребовались бы повторные прогоны агентов по многим вариантам — за рамками одной конфигурации, отражённой выше.

9. Ограничения

Несколько ограничений очерчивают то, что эти результаты подтверждают. Публичные случаи компактны — сети из нескольких узлов и короткие временные ряды, — поэтому они проверяют, владеет ли агент методом, а не масштабируется ли он на промышленные объёмы данных. Оценщики являются детерминированными суррогатами, и никаких утверждений о промышленной точности в переменном токе не оправдано, пока не появятся проверки на основе симулятора. При одном случае на семейство на каждый набор ранжирование по 41 точке статистически хрупко, хотя генератор делает увеличение выборки недорогим. Задачи также самодостаточны: агент вычисляет каждый ответ напрямую и ни разу не обязан управлять профессиональным программным обеспечением для энергосистем, поэтому бенчмарк пока не измеряет владение симуляторами и инструментами, от которых зависит значительная часть реальной инженерной работы. Эталонное решение получает единицу по построению, что является проверкой самосогласованности контракта, а не нижней границей сложности, и не может удостоверить оценщик. Метод контроля качества из раздела 8.2 — чтение единодушного провала агентов как вероятного дефекта — имеет предел, который растёт со сложностью бенчмарка: он может отличить сломанную задачу от просто трудной лишь пока агенты достаточно сильны, чтобы подлинные провалы были редкостью, так что по мере приближения задач к переднему краю и учащения

честных провалов единодушный провал перестаёт ясно указывать на спецификацию, и нерешённый хвост приходится подтверждать независимым пересчётом, а не согласием агентов. И поскольку каждый балл сообщается относительно эталонного оптимума задачи, шкала надёжна ровно настолько, насколько оптимален этот эталон: мы проверяем оптимальность при порождении скрытого случая и трактуем любое решение, превосходящее эталон, как сигнал, а не как балл выше единицы, но мы не доказываем глобальную оптимальность для каждого семейства, поэтому любая будущая задача с непрерывной целевой величиной потребовала бы установить её эталон с той же тщательностью. Зерно скрытого тура расходуется, как только тур раскрыт, поэтому цитируемая таблица лидеров требует свежего зерна на изолированном хосте. Песочница изолирует перемещением, а не контейнеризацией, что устраняет утечку через относительные пути и окружение, но не целенаправленный поиск оценщика по абсолютному пути, поэтому состязательное использование потребовало бы контейнеров уровня хоста; упаковка бенчмарка как окружения для контейнерной обвязки оценивания вроде Harbor [harbor2026] — естественный путь к такой изоляции, и она дополнительно дала бы интерфейсы параллельного исполнения и роллаутов (rollouts), нужные для оценки дисперсии выборки модели, которую мы сейчас оставляем неизмеренной. Мы прогоняем по одной конфигурации на агента и поэтому не оцениваем дисперсию выборки модели.

Более широкий вопрос — чего стоит бенчмарк, когда сильнейшие нынешние агенты уже решают большинство его публичных задач. На этом компактном уровне его способность ранжировать ведущих агентов ограничена, и мы не представляем его как таблицу лидеров, ставящую в тупик передний край; его ценность лежит в другом. Он разделяет реалистичный спектр агентов, а не только лидеров, причём OpenCode, на небольшой открытой модели, явно отстаёт от переднего края и полностью проваливается там, где лидеры справляются. Он вскрывает общие слепые зоны даже при высоких баллах, потому что задача, которую каждый агент упускает одинаково, — это пробел в способностях, который исполняемая проверка делает видимым, а текстовое оценивание — нет. Его контракт устроен так, чтобы прорасти в более трудные уровни раздела 10, где агент должен управлять профессиональными симуляторами, а не вычислять компактные экземпляры, и где запас велик. А его устойчивость к загрязнению проспективна: генератор сохраняет доступной незагрязнённую выборку по мере того, как публичный набор стареет и на нём обучаются. Вклад — это растущая, заземлённая на физику, устойчивая к загрязнению мера для области, имевшей лишь текстовое оценивание, а не одноразовая стена сложности.

10. Заключение

Мы перенесли исполняемое оценивание, уже ставшее стандартом для агентов в программной инженерии и работе с инструментами, в электроэнергетику, где оценивание оставалось текстовым. Бенчмарк превращает задачи энергосистем в проверки, пересчитывающие физику; на матрице нынешних агентов сильнейшие уже набирают близко к максимуму на нынешнем компактном уровне, тогда как OpenCode, меньшая открытая модель, отстаёт, а результаты на публичном и скрытом наборах согласуются. Его наиболее переносимый вклад — не какая-либо отдельная задача, а метод: чтение единодушного провала как сигнала о дефекте, который по ходу этой работы неоднократно вскрывал дефекты спецификаций и даже латентную ошибку в собственной физике оценщика, которую бенчмарк не смог бы поймать одной лишь самосогласованностью, — напоминание о том, что эталонный ответ, выведенный из оценщика, не может его удостоверить. Долговременная ценность артефакта — в контракте, заземлённом на предметную область наборе задач, оценщиках, генераторе скрытого набора на основе зерна с его проспективной устойчивостью к загрязнению, изолированном раннере и воспроизводимой процедуре контроля качества; одно из направлений дальнейшей работы — наполнить оценщики реализациями на основе симулятора, не нарушая контракт, который их скрепляет.

Вторая ось роста касается агента, а не оценщика. Представленные здесь задачи самодостаточны: агент читает случай и вычисляет ответ, и хотя он может писать и запускать вспомогательный код, ни одна задача пока не требует от него управлять профессиональным программным обеспечением для энергосистем. Между тем значительная часть реальной инженерной работы — это именно оно: управление пакетом расчёта потокораспределения, инструментом расчёта КЗ или динамическим симулятором, — и появляющиеся ныне в отрасли эксплуатационные агенты уже оркеструют такие инструменты. Контракт здесь не накладывает ограничений: агент волен вызывать любые инструменты, которые предоставляет его песочница, чтобы получить решение. Поэтому естественное следующее поколение задач измеряло бы не компетентность в замкнутой форме, а владение этими специализированными средами, где симулятор служит инструментом агента, а не оценщика. Двойственность стоит назвать: один и тот же симулятор может стоять по любую сторону контракта — как инструмент агента для решения или как бэкенд оценщика для проверки, — и изоляция из раздела 4.2 — это ровно то, что должно держать их порознь, так чтобы агент никогда не мог добраться до проверяющего экземпляра как до оракула.

Литература

- **[abur2004se]** Abur, Ali; Expósito, Antonio Gómez. *Power System State Estimation: Theory and Implementation*. Marcel Dekker, 2004.
- **[agenticpowersurvey2025]** *Agentic AI Systems in Electrical Power Systems Engineering: Current State-of-the-Art and Challenges*. arXiv preprint arXiv:2511.14478, 2025.
- **[aisjudgeais2025]** *When AIs Judge AIs: The Rise of Agent-as-a-Judge Evaluation for LLMs*. arXiv preprint arXiv:2508.02994, 2025.
- **[anderson1995faulted]** Anderson, Paul M. *Analysis of Faulted Power Systems*. Wiley-IEEE Press, 1995. ISBN 978-0-7803-1145-9.
- **[babaeinejadsarookolaee2019pglib]** Babaeinejadsarookolaee, Sogol et al. *The Power Grid Library for Benchmarking AC Optimal Power Flow Algorithms*. arXiv preprint arXiv:1908.02788, 2019.
- **[brown2018pypsa]** Brown, Tom; Hörsch, Jonas; Schlachtberger, David. *PyPSA: Python for Power System Analysis*. Journal of Open Research Software 6, 2018.
- **[brown2024monkeys]** Brown, Bradley; Juravsky, Jordan; Ehrlich, Ryan; Clark, Ronald; Le, Quoc V.; Ré, Christopher; Mirhoseini, Azalia. *Large Language Monkeys: Scaling Inference Compute with Repeated Sampling*. arXiv preprint arXiv:2407.21787, 2024.
- **[chan2024mlebench]** Chan, Jun Shern et al. *MLE-bench: Evaluating Machine Learning Agents on Machine Learning Engineering*. arXiv preprint arXiv:2410.07095, 2024.
- **[chen2021codex]** Chen, Mark et al. *Evaluating Large Language Models Trained on Code*. arXiv preprint arXiv:2107.03374, 2021.
- **[coffrin2018powermodels]** Coffrin, Carleton; Bent, Russell; Sundar, Kaarthik; Ng, Yeesian; Lubin, Miles. *PowerModels.jl: An Open-Source Framework for Exploring Power Flow Formulations*. Power Systems Computation Conference (PSCC), 2018.
- **[contaminationsurvey2025]** *A Survey on Data Contamination for Large Language Models*. arXiv preprint arXiv:2502.14425, 2025.
- **[cui2021andes]** Cui, Hantao; Li, Fangxing; Tomsovic, Kevin. *Hybrid Symbolic-Numeric Framework for Power System Modeling and Analysis*. IEEE Transactions on Power Systems 36, 1373–1384, 2021.
- **[dugan2011opendss]** Dugan, Roger C.; McDermott, Thomas E. *An Open Source Platform for Collaborating on Smart Grid Research*. IEEE PES General Meeting, 2011.
- **[en50160]** *EN 50160:2010 Voltage characteristics of electricity supplied by public distribution networks*. CENELEC, 2010.
- **[entsoerfg]** *ENTSO-E Network Code on Requirements for Grid Connection of Generators (RfG)*.

ENTSO-E / European Commission Regulation 2016/631, 2016.
- **[gridmind2025]** *GridMind: LLMs-Powered Agents for Power System Analysis and Operations*. arXiv preprint arXiv:2509.02494, 2025.
- **[gridmindcia2026]** *Grid-Mind: An LLM-Orchestrated Multi-Fidelity Agent for Automated Connection Impact Assessment*. arXiv preprint arXiv:2602.20683, 2026.
- **[harbor2026]** Terminal-Bench Team. *Harbor: A Framework for Running Agent Evaluations and RL Environments*. https://github.com/harbor-framework/harbor, 2026. Container-based agent-evaluation harness released with Terminal-Bench 2.0.
- **[horowitz2014relaying]** Horowitz, Stanley H.; Phadke, Arun G. *Power System Relaying*. 4th ed., Wiley, 2014. ISBN 978-1-118-66200-7.
- **[iec600767]** *IEC 60076-7:2018 Power transformers — Part 7: Loading guide for mineral-oil-immersed power transformers*. IEC, 2018.
- **[iec60364552]** *IEC 60364-5-52 Low-voltage electrical installations — Selection and erection of electrical equipment — Wiring systems*. IEC, 2009.
- **[iec60909]** *IEC 60909-0:2016 Short-circuit currents in three-phase a.c. systems — Part 0: Calculation of currents*. IEC, 2016.
- **[ieee1366]** *IEEE Std 1366-2012 Guide for Electric Power Distribution Reliability Indices*. IEEE, 2012.
- **[ieee2800]** *IEEE Std 2800-2022 Standard for Interconnection and Interoperability of Inverter-Based Resources Interconnecting with Associated Transmission Electric Power Systems*. IEEE, 2022.
- **[ieee519]** *IEEE Std 519-2014 Recommended Practice and Requirements for Harmonic Control in Electric Power Systems*. IEEE, 2014.
- **[ieee738]** *IEEE Std 738-2012 Standard for Calculating the Current-Temperature Relationship of Bare Overhead Conductors*. IEEE, 2012.
- **[jain2024livecodebench]** Jain, Naman et al. *LiveCodeBench: Holistic and Contamination Free Evaluation of Large Language Models for Code*. arXiv preprint arXiv:2403.07974, 2024.
- **[jimenez2024swebench]** Jimenez, Carlos E.; Yang, John; Wettig, Alexander; Yao, Shunyu; Pei, Kexin; Press, Ofir; Narasimhan, Karthik. *SWE-bench: Can Language Models Resolve Real-World GitHub Issues?*. ICLR, 2024. arXiv:2310.06770.
- **[kapoor2024agentsmatter]** Kapoor, Sayash; Stroebl, Benedikt; Siegel, Zachary S.; Nadgir, Nitya; Narayanan, Arvind. *AI Agents That Matter*. arXiv preprint arXiv:2407.01502, 2024.
- **[kiela2021dynabench]** Kiela, Douwe et al. *Dynabench: Rethinking Benchmarking in NLP*. NAACL, 2021.
- **[kundur1994stability]** Kundur, Prabha. *Power System Stability and Control*. McGraw-Hill, 1994.
- **[liu2011fdi]** Liu, Yao; Ning, Peng; Reiter, Michael K. *False Data Injection Attacks against State Estimation in Electric Power Grids*. ACM Transactions on Information and System Security 14, 2011. First presented at ACM CCS 2009.
- **[liu2024agentbench]** Liu, Xiao et al. *AgentBench: Evaluating LLMs as Agents*. ICLR, 2024. arXiv:2308.03688.
- **[llmpowersurvey2025]** *Large Language Models for Power System Applications: A Comprehensive Literature Survey*. arXiv preprint arXiv:2512.13004, 2025.
- **[marot2020l2rpn]** Marot, Antoine; Donnot, Benjamin; Romero, Camilo; Donon, Balthazar; Lerousseau, Marvin; Veyrin-Forrer, Luca; Guyon, Isabelle. *Learning to Run a Power Network Challenge for Training Topology Controllers*. Electric Power Systems Research 189, 2020. arXiv:1912.04211.
- **[marot2021retrospective]** Marot, Antoine; Donnot, Benjamin; Dulac-Arnold, Gabriel; Kelly, Adrian; O'Sullivan, Aidan; Viebahn, Jan; Awad, Mariette; Guyon, Isabelle; Panciatici, Patrick; Romero, Camilo. *Learning to Run a Power Network Challenge: a Retrospective Analysis*. NeurIPS 2020 Competition and Demonstration Track, PMLR v133, 112–132, 2021.

- **[mialon2023gaia]** Mialon, Grégoire; Fourrier, Clémentine; Swift, Craig; Wolf, Thomas; LeCun, Yann; Scialom, Thomas. *GAIA: A Benchmark for General AI Assistants*. arXiv preprint arXiv:2311.12983, 2023.
- **[pfagent2026]** *PFAgent: A Tractable and Self-Evolving Power-Flow Agent for Interactive Grid Analysis*. arXiv preprint arXiv:2604.10846, 2026.
- **[pfbench2025]** *PFBench: Power-Flow Benchmark for LLM-based Power System Agent Evaluation*. IEEE DataPort, 2025. https://ieee-dataport.org/documents/power-flow-benchmark-llm-based-power-system-agent-evaluation-pfbench
- **[poweragentbench]** PowerAgent community, Harvard SEAS. *PowerAgentBench: Standardized Tasks, Environments, and Metrics for Power-System Agents*. GitHub repository, Power-Agent/PowerAgentBench, 2026. Benchmark component of the PowerAgent ecosystem (poweragent.seas.harvard.edu); no dedicated publication at the time of writing.
- **[restoration2020]** *A Two-Stage Service Restoration Method for Electric Power Distribution Systems*. arXiv preprint arXiv:2004.07921, 2020.
- **[scuccts2020]** *Security Constrained Unit Commitment with Corrective Transmission Switching*. arXiv preprint arXiv:2001.00597, 2020.
- **[terminalbench2025]** Laude Institute; Stanford University. *Terminal-Bench: Benchmarking AI Agents on Realistic Terminal Tasks*. https://www.tbench.ai/, 2025.
- **[thurner2018pandapower]** Thurner, Leon; Scheidler, Alexander; Schäfer, Florian; Menke, Jan-Hendrik; Dollichon, Julian; Meier, Friederike; Meinecke, Steffen; Braun, Martin. *pandapower — An Open-Source Python Tool for Convenient Modeling, Analysis, and Optimization of Electric Power Systems*. IEEE Transactions on Power Systems 33, 6510–6521, 2018.
- **[turitsyn2010voltvar]** Turitsyn, Konstantin; v Sulc, Petr; Backhaus, Scott; Chertkov, Michael. *Local Control of Reactive Power by Distributed Photovoltaic Generators*. arXiv preprint arXiv:1006.0160, 2010.
- **[wang2023fairevaluators]** Wang, Peiyi et al. *Large Language Models are not Fair Evaluators*. arXiv preprint arXiv:2305.17926, 2023.
- **[white2024livebench]** White, Colin et al. *LiveBench: A Challenging, Contamination-Free LLM Benchmark*. arXiv preprint arXiv:2406.19314, 2024.
- **[xgridagent2025]** *X-GridAgent: An LLM-Powered Agentic AI System for Assisting Power Grid Analysis*. arXiv preprint arXiv:2512.20789, 2025.
- **[yao2024taubench]** Yao, Shunyu; Shinn, Noah; Razavi, Pedram; Narasimhan, Karthik. *τ-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains*. arXiv preprint arXiv:2406.12045, 2024.
- **[zhang2025poweragent]** Zhang, Qian; Xie, Le. *PowerAgent: A Road Map Toward Agentic Intelligence in Power Systems: Foundation Model, Model Context Protocol, and Workflow*. IEEE Power & Energy Magazine 23, 93–101, 2025.
- **[zheng2023judging]** Zheng, Lianmin et al. *Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena*. NeurIPS Datasets and Benchmarks, 2023. arXiv:2306.05685.
- **[zhou2019contingency]** *Graph Computing based Fast Screening in Contingency Analysis*. arXiv preprint arXiv:1904.00044, 2019.
- **[zhou2024webarena]** Zhou, Shuyan et al. *WebArena: A Realistic Web Environment for Building Autonomous Agents*. ICLR, 2024. arXiv:2307.13854.
- **[zhou2025elecbench]** Zhou, Xiyuan et al. *ElecBench: a Power Dispatch Evaluation Benchmark for Large Language Models*. IEEE PES General Meeting, 2025. arXiv:2407.05365; Best Paper.
- **[zhuge2024agentjudge]** Zhuge, Mingchen; Zhao, Changsheng; Ashley, Dylan; Wang, Wenyi; Khizbullin, Dmitrii; Xiong, Yunyang; Liu, Zechun; Chang, Ernie; Krishnamoorthi, Raghuraman; Tian, Yuandong; Shi, Yangyang; Chandra, Vikas; Schmidhuber, Jürgen. *Agent-as-a-Judge: Evaluate*

Agents with Agents. arXiv preprint arXiv:2410.10934, 2024.
- **[zimmerman2011matpower]** Zimmerman, Ray D.; Murillo-Sánchez, Carlos E.; Thomas, Robert J. *MATPOWER: Steady-State Operations, Planning, and Analysis Tools for Power Systems Research and Education*. IEEE Transactions on Power Systems 26, 12–19, 2011.

Приложение A. Каталог задач

Ниже перечислены 41 семейство по предметным областям, каждое — со своим первичным источником или определяющим стандартом и уровнем достоверности из раздела 5.1 (11 высоких, 28 средних, 2 низких). Бенчмарк поставляет полную запись по каждому семейству на английском и русском: полный контракт задачи — входные данные, цель, жёсткие ограничения и схему решения — вместе с записью о происхождении (название источника, локатор и заметка, отображающая источник на исполняемый случай).

1. Анализ сети и эксплуатационные ограничения

Семейство	Источник / стандарт	Достоверность
`ybus_construction`	Glover, Overbye & Sarma (учебник)	высокая
`pf_voltage_repair`	Glover, Overbye & Sarma (учебник)	высокая
`contingency_screening`	Скрининг аварийных ситуаций на графовых вычислениях	средняя
`corrective_transmission_switching`	SCLIC с корректирующими переключениями (arXiv:2001.00597)	средняя
`dynamic_thermal_rating`	IEEE 738; Safari et al., вероятностный DTR	средняя

2. Короткое замыкание и релейная защита

Семейство	Источник / стандарт	Достоверность
`three_phase_short_circuit`	IEC 60909	высокая
`earth_fault_calculation`	Anderson, Analysis of Faulted Power Systems	средняя
`breaker_relay_short_circuit`	IEC 60909; Glover, Overbye & Sarma	высокая
`distance_protection_settings`	Horowitz & Phadke, Power System Relaying	средняя
`overcurrent_protection`	Horowitz & Phadke, Power System Relaying	средняя
`cable_ampacity_sizing`	IEC 60364-5-52 / IEC 60949	высокая

3. Устойчивость, сетевой кодекс и инверторные источники

Семейство	Источник / стандарт	Достоверность
`critical_clearing_time`	Критерий равных площадей (Kundur)	высокая
`transient_stability_prediction`	Chen et al., прогноз переходной устойчивости	средняя
`frt_compliance`	IEEE 2800 / ENTSO-E RfG (FRT)	средняя
`ibr_short_circuit_frt`	Моделирование инверторных источников для КЗ / FRT	средняя
`min_synchronous_share`	Wen et al., устойчивость частоты при низкой инерции	низкая

4. Распределённые ресурсы, СЭС, электромобили и накопители

Семейство	Источник / стандарт	Достоверность
`pv_volt_var`	Turitsyn et al., локальное управление Volt-VAR	средняя
`ev_v2g_outage_schedule`	Электромобили для качества и безопасности электроснабжения	средняя
`ev_v2g_voltage_support`	Электромобили для качества и безопасности электроснабжения	средняя
`bess_ancillary_response`	Gonzalez-Longatt & Rueda Torres	средняя
`commercial_pv_lcoe_uncertainty`	Неопределённость «мягких» затрат СЭС (NREL)	средняя

5. Микросети и диспетчеризация

Семейство	Источник / стандарт	Достоверность
`microgrid_economic_dispatch`	España et al., диспетчеризация микросети	средняя
`rolling_microgrid_dispatch`	España et al., диспетчеризация микросети	средняя
`islanded_microgrid_pq_dispatch`	España et al., диспетчеризация микросети	средняя
`dispatch_uncertainty`	Chung, продвинутое прогнозирование для умных сетей	низкая
`hydro_thermal_storage_uc`	Выбор состава оборудования методом ADMM	средняя

Семейство	Источник / стандарт	Достоверность
n1_generation_commitment	SCUC с корректирующими переключениями (arXiv:2001.00597)	средняя

6. Надёжность и восстановление

Семейство	Источник / стандарт	Достоверность
flisr_restoration	Двухэтапное восстановление электроснабжения в распределительной сети	средняя
fault_section_localization	Brown, индикаторы повреждения цепи	средняя
fci_placement	Brown, индикаторы повреждения цепи	средняя
fci_saidi_caidi	Brown, индикаторы повреждения; показатели IEEE 1366	средняя
operator_breaker_load_actions	Glover, Overbye & Sarma (учебник)	средняя

7. Качество электроэнергии, стандарты, активы и кибербезопасность

Семейство	Источник / стандарт	Достоверность
en50160_voltage_compliance	EN 50160	высокая
power_quality_event_classification	IEC 61000-4-30	высокая
harmonic_ieee519_compliance	IEEE 519	высокая
transformer_thermal_loading	IEC 60076-7	высокая
fdi_state_estimation	Abur & Exposito, оценивание состояния	высокая
protected_meter_placement	Графическая защита от инъекции недостоверных данных	средняя
mas_frequency_handover	Многоагентное управление активными распределительными сетями	средняя

8. Прогнозирование при неопределённости

Семейство	Источник / стандарт	Достоверность
wind_power_forecast	Safari et al., краткосрочный прогноз ветра	средняя

Семейство	Источник / стандарт	Достоверность
`wind_prediction_interval`	Khorramdel et al., нечёткие доверительные интервалы	средняя

Приложение В. Артефакты эксперимента

Конфигурация прогона. Каждый агент вызывался по одному разу на случай через собственный интерфейс командной строки при времени ожидания 600 секунд на случай и политике инфраструктурных перезапусков из раздела 4.2 — не более двух попыток, с перезапуском только при кратковременных сбоях провайдера или транспорта, но никогда — при провалах качества, контракта или по времени. На каждое семейство прогонялись один публичный случай и один свежесгенерированный скрытый случай, и каждое решение оценивалось детерминированными, нормированными на оптимум оценщиками из раздела 6. Версии: Codex на gpt-5.5 со средним уровнем рассуждения (codex-cli 0.139.0); агент командной строки Cursor на composer-2.5 (cursor-agent 2026.06.12-01-15-52-7244546); OpenCode на deepseek-v4-flash-free (opencode 1.15.13).

Недоборы на публичном наборе. Каждый публичный случай, не набравший единицу, таков. Codex недобрал на `contingency_screening` (0.9) и вернул недопустимый `ev_v2g_outage_schedule`, при этом `pv_volt_var` оказался в пределах 10^{-5} от оптимума, но не дотянул до точной единицы; Cursor недобрал на `contingency_screening` (0.9) и `ev_v2g_outage_schedule` (недопустимо); OpenCode вернул недопустимые решения для `ev_v2g_outage_schedule` и `pv_volt_var`. Все остальные публичные случаи набрали единицу.

Распространение. Эталонный тур поставляется с бенчмарком в виде по-агентных, по-наборных сводок прогонов. Для публичного набора включаются также по-кейсные решения, результаты оценщика и журналы агента; для скрытого набора публикуются только агрегированные сводки, поскольку по-кейсное скрытое решение или результат оценщика раскрыли бы ответ этого случая (раздел 4.4).